

Gradient Boosting: Complete Guide

Including XGBoost, LightGBM & CatBoost

Sourav Karmakar

Outline

- 1 Introduction
- 2 AdaBoost vs Gradient Boost
- 3 Gradient Boost for Regression
- 4 Gradient Boost for Classification
- 5 Mathematical Foundation
- 6 XGBoost (Extreme Gradient Boosting)
- 7 LightGBM
- 8 CatBoost
- 9 Comprehensive Comparison
- 10 Key Takeaways

Introduction

What is Gradient Boosting?

Gradient Boosting is one of the most popular machine learning algorithms — often called **Data Science's silver bullet** for its outstanding real-world performance.

Definition

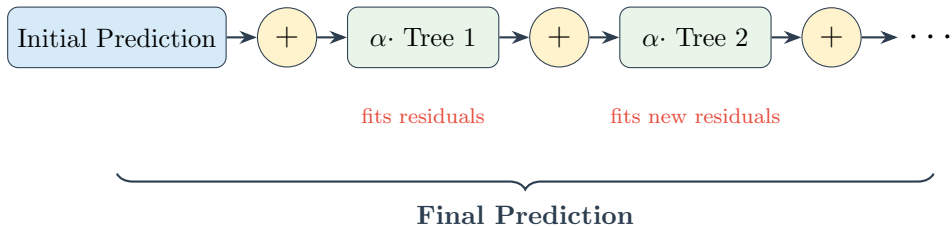
Gradient Boosting is a machine learning technique that builds an **ensemble of weak learners**, typically decision trees, in a **stage-wise** fashion to minimize a loss function. The original algorithm is often referred to as **GBM** (Gradient Boosted Machine).

In Practice

We rarely use the basic GBM directly; instead we rely on optimized libraries: **XGBoost**, **LightGBM**, or **CatBoost**.

Prerequisites: Classification & Regression Trees, AdaBoost, Regularization.

Core Idea of Gradient Boosting



Each new tree corrects the errors (residuals) of the combined ensemble so far.

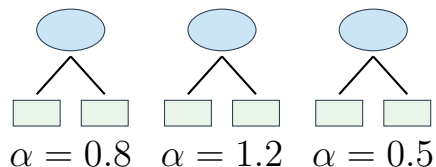
AdaBoost vs Gradient Boost

AdaBoost vs Gradient Boost

Aspect	AdaBoost	Gradient Boost
Initial Prediction	Starts with a stump (very short tree)	Starts with a single leaf (initial guess)
Tree Size	Usually stumps (2 leaves)	Larger trees (4–32 leaves typically)
Scaling	Each tree scaled by different amounts based on performance	All trees scaled by the same learning rate
Tree Building	Based on errors from previous stumps	Based on residuals from previous trees

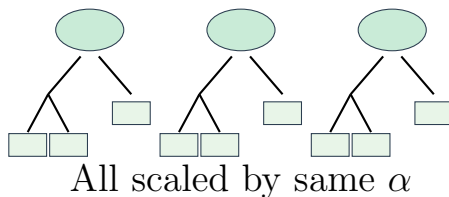
AdaBoost vs Gradient Boost – Visual Comparison

AdaBoost



Different weights!

Gradient Boost



Same weight!

Gradient Boost for Regression

Training Data Example

Predict **Weight** based on Height, Favorite Color, and Gender:

Person	Height (cm)	Fav. Color	Gender	Weight (kg)
1	180	Blue	Male	88
2	170	Red	Female	76
3	175	Green	Male	80
4	160	Blue	Female	70
5	165	Red	Male	65
6	155	Green	Female	48

Step 1: Initial Prediction

For regression, the first guess is the **average** of all target values:

$$F_0 = \frac{88 + 76 + 80 + 70 + 65 + 48}{6} = \mathbf{71.2 \text{ kg}}$$

Initial Leaf

71.2 kg

Everyone gets the same prediction: **71.2 kg**.

Step 2: Calculate Pseudo-Residuals

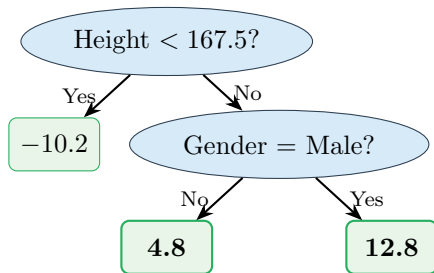
Pseudo-residuals measure the gap between observed and predicted:

$$r_i = y_i - F_0(x_i) = y_i - 71.2$$

Person	Observed	Predicted	Pseudo-Residual
1	88	71.2	+16.8
2	76	71.2	+4.8
3	80	71.2	+8.8
4	70	71.2	-1.2
5	65	71.2	-6.2
6	48	71.2	-23.2

Step 3: Build First Tree on Residuals

Build a regression tree to **predict the pseudo-residuals** using the input features:



Note: We limit trees to ≤ 4 leaves here. In practice, 8–32 leaves are common.

Step 4: Apply Learning Rate

To prevent overfitting, we scale the tree's contribution with a **learning rate**:

Update Rule

$$F_1(\mathbf{x}) = F_0(\mathbf{x}) + \alpha \cdot h_1(\mathbf{x}), \quad \alpha = 0.1$$

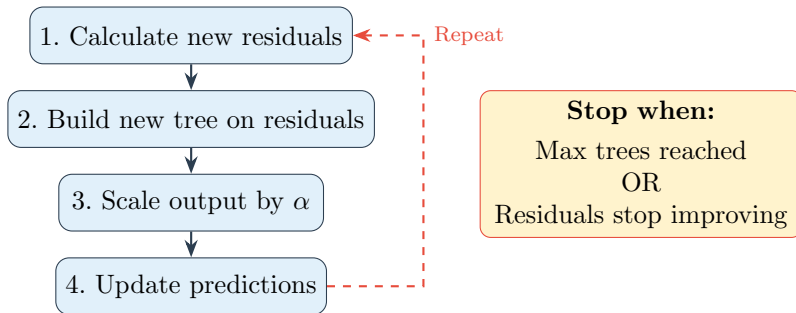
Example – Person 1 (Height=180, Male $\Rightarrow$ tree output = 12.8):

$$F_1 = 71.2 + (0.1 \times 12.8) = 71.2 + 1.28 = \mathbf{72.48 \text{ kg}}$$

Key Insight

Empirical evidence shows that taking **lots of small steps** in the right direction results in better predictions with testing data (lower variance).

Step 5: Iterate Until Convergence



Final Prediction for Regression

For new data, combine all trees:

Gradient Boost Regression – Final Model

$$F_M(\mathbf{x}) = F_0 + \alpha \sum_{m=1}^M h_m(\mathbf{x})$$

where α is the learning rate and h_m is the m -th tree.

$$\underbrace{F_0}_{\text{mean}} + \underbrace{\alpha \cdot h_1}_{\text{}} + \underbrace{\alpha \cdot h_2}_{\text{}} + \cdots + \underbrace{\alpha \cdot h_M}_{\text{}} = \underbrace{F_M(\mathbf{x})}_{\text{prediction}}$$

Gradient Boost for Classification

Training Data Example

Classify whether a customer **will purchase** a product:

Customer	Age	Income (\$K)	Credit Card	Purchase?
1	25	45	Yes	Yes
2	35	55	No	No
3	45	75	Yes	Yes
4	28	35	No	No
5	40	80	Yes	Yes
6	32	60	Yes	Yes

4 purchased, 2 did not $\Rightarrow$ imbalanced toward “Yes”.

Step 1: Initial Prediction (Log-Odds)

For classification, the initial prediction is the **log-odds**:

$$F_0 = \log\left(\frac{\#Yes}{\#No}\right) = \log\left(\frac{4}{2}\right) = \mathbf{0.7}$$

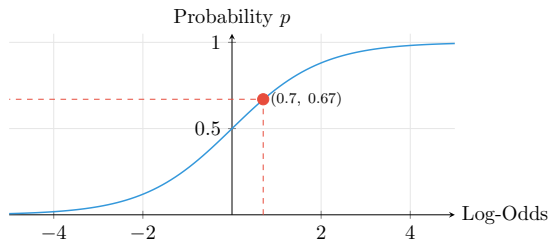
Initial Leaf (log-odds)

0.7

Step 2: Convert Log-Odds to Probability

Use the **logistic function** (sigmoid) to convert:

$$p = \frac{e^{\text{log-odds}}}{1 + e^{\text{log-odds}}} = \frac{e^{0.7}}{1 + e^{0.7}} \approx \mathbf{0.67}$$



Since $0.67 > 0.5$, we initially classify **everyone** as “Will Purchase”.

Step 3: Calculate Pseudo-Residuals

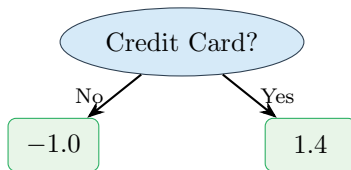
Residuals = Observed – Predicted Probability:

Customer	Observed (0/1)	Predicted Prob	Pseudo-Residual
1	1	0.67	+0.33
2	0	0.67	-0.67
3	1	0.67	+0.33
4	0	0.67	-0.67
5	1	0.67	+0.33
6	1	0.67	+0.33

Positive residuals: model underestimates. Negative: model overestimates.

Step 4: Build Tree & Output Value Transformation

Build a tree on residuals, but leaves need a **special transformation**:



Output Value Formula for Classification

$$\text{Output Value} = \frac{\sum \text{Residuals in Leaf}}{\sum p_{\text{prev}}(1 - p_{\text{prev}})}$$

Output Value Calculation Example

Right leaf (4 customers with credit cards, residuals: 0.33, 0.33, 0.33, 0.33):

$$\begin{aligned}\text{Output} &= \frac{0.33 + 0.33 + 0.33 + 0.33}{4 \times 0.67 \times (1 - 0.67)} \\ &= \frac{1.32}{4 \times 0.67 \times 0.33} = \frac{1.32}{0.88} \approx 1.4\end{aligned}$$

Why the special formula?

Predictions are in **log-odds** space, but residuals come from **probability** space. The denominator $\sum p(1 - p)$ transforms them correctly — it is the **second derivative** (Hessian) of the log-loss.

Step 5: Update Predictions

Combine the initial leaf with the scaled tree output (learning rate $\alpha = 0.8$):

$$\text{New Log-Odds} = F_0 + \alpha \cdot h_1(\mathbf{x})$$

For a customer with credit card (tree output = 1.4):

$$\text{New Log-Odds} = 0.7 + (0.8 \times 1.4) = 0.7 + 1.12 = \mathbf{1.82}$$

Convert back to probability:

$$p = \frac{e^{1.82}}{1 + e^{1.82}} \approx \mathbf{0.86}$$

Note: $\alpha = 0.8$ is used for illustration. In practice, $\alpha \approx 0.1$ is typical.

Final Classification

For new data, combine all components:

Final Model

$$\text{Final Log-Odds} = F_0 + \alpha \sum_{m=1}^M h_m(\mathbf{x})$$

Convert to probability and apply threshold:

$$p = \frac{e^{\text{log-odds}}}{1 + e^{\text{log-odds}}}$$

$p > 0.5 \Rightarrow$ **Purchase**

$p \leq 0.5 \Rightarrow$ **No Purchase**

Mathematical Foundation

General Gradient Boosting Framework

Objective: Minimize loss function $L(y, F(\mathbf{x}))$

Loss Functions:

- **Regression** (Squared Error): $L(y, F) = \frac{1}{2}(y - F)^2$
- **Classification** (Log Loss, $y \in \{0, 1\}$):

$$L(y, F) = -[y \log(p) + (1 - y) \log(1 - p)], \quad p = \frac{1}{1 + e^{-F}}$$

Pseudo-Residuals as Negative Gradients

The pseudo-residual for sample i at iteration m is the **negative gradient** of the loss:

$$r_{im} = - \left. \frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right|_{F=F_{m-1}}$$

Regression ($L = \frac{1}{2}(y - F)^2$)

$$r_{im} = y_i - F_{m-1}(x_i)$$

Exactly the **residual**!

Classification (Log Loss)

$$r_{im} = y_i - p_{m-1}(x_i)$$

Observed – predicted probability.

This is why we call them **pseudo**-residuals — they are residuals only for squared loss; for other losses they are negative gradients that give the direction to reduce the loss.

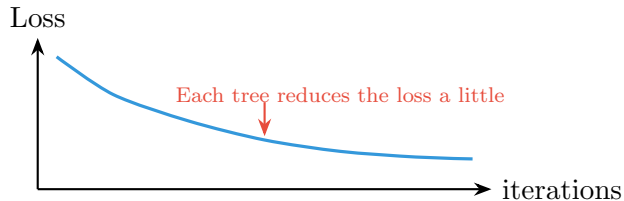
Iterative Update Rule

Model Update

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \alpha \cdot h_m(\mathbf{x})$$

where:

- $F_m(\mathbf{x})$ = prediction after m iterations
- α = learning rate (typically 0.1)
- $h_m(\mathbf{x})$ = m -th tree fitted to pseudo-residuals



Complete Gradient Boosting Algorithm

Algorithm: Gradient Boosting

Input: $\{(x_i, y_i)\}_{i=1}^n$ (Data), L (Loss), M (Trees), α (Learning Rate)

- 1 Initialize: $F_0(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$
- 2 For $m = 1$ to M :
 - 2a. Residuals: $r_{im} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}$
 - 2b. Fit tree h_m to targets $\{r_{im}\}$
 - 2c. Leaf weights: $\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$
 - 2d. Update: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \alpha \sum_j \gamma_{jm} \mathbf{1}(x \in R_{jm})$
- 3 Output: $F_M(\mathbf{x})$

Notation Key:

- γ : The predicted value (step size) that minimizes loss.
- r_{im} : Pseudo-residuals (negative gradient of the loss).
- R_{jm} : The j -th terminal node (leaf) of the m -th tree.
- $\mathbf{1}(x \in R_{jm})$: Indicator function (1 if x is in leaf j , else 0).

XGBoost (Extreme Gradient Boosting)

What is XGBoost?

XGBoost (Extreme Gradient Boosting) is an optimized gradient boosting library by **Tianqi Chen**, designed for efficiency, flexibility, and portability.

Key Differences from Standard GBM

- **Regularization:** Built-in L1 and L2 regularization
- **Second-Order Optimization:** Uses both gradients and Hessians (Newton's method)
- **System Optimizations:** Column block storage, cache-aware access
- **Sparsity Handling:** Efficient handling of missing values
- **Parallelization:** Parallel split finding (not parallel trees)
- **Tree Pruning:** Grows to max_depth, then prunes backward

XGBoost Objective Function

XGBoost minimizes a **regularized** objective:

$$\mathcal{L}(\phi) = \underbrace{\sum_{i=1}^n l(y_i, \hat{y}_i)}_{\text{Training Loss}} + \underbrace{\sum_{k=1}^K \Omega(f_k)}_{\text{Regularization}}$$

where the regularization term is:

$$\Omega(f_k) = \gamma T_k + \frac{1}{2} \lambda \sum_{j=1}^{T_k} w_j^2 + \alpha \sum_{j=1}^{T_k} |w_j|$$

- T_k = number of leaves in tree f_k
- w_j = leaf weight (prediction value for leaf j)
- γ = minimum split loss (tree complexity penalty)
- λ = L2 regularization, α = L1 regularization

Second-Order Taylor Approximation

XGBoost approximates the loss using a **2nd-order Taylor expansion**:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

where:

$$g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \quad (\text{gradient}), \quad h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2} \quad (\text{Hessian})$$

Why second-order?

Using both gradient **and** curvature (Hessian) allows XGBoost to take more informed optimization steps — similar to Newton's method vs. plain gradient descent.

Optimal Leaf Weights and Objective

Removing constant terms and grouping by leaves:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T$$

Setting derivative w.r.t. w_j to zero:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

And the optimal objective (quality score):

$$\text{Obj}^* = - \frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

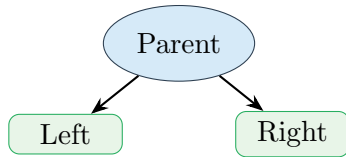
Similarity Score and Gain

For **regression** ($h_i = 1$ for squared loss), the optimal leaf weight simplifies to:

$$\text{Similarity Score} = \frac{(\sum \text{Residuals})^2}{\text{Number of Residuals} + \lambda}$$

The **Gain** of a split measures improvement:

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

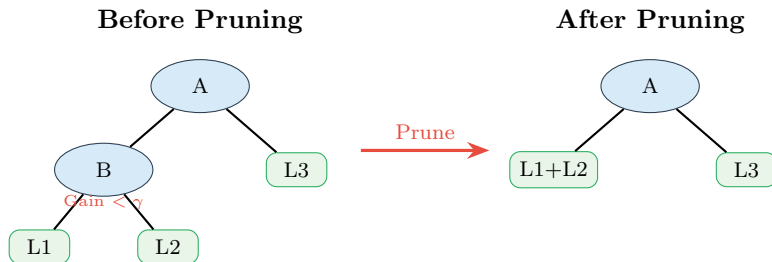


$$\text{Gain} = \text{SS}_{\text{Left}} + \text{SS}_{\text{Right}} - \text{SS}_{\text{Parent}} - \gamma$$

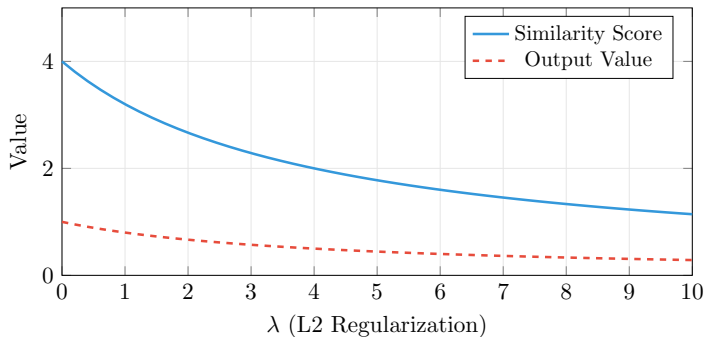
Pruning in XGBoost

XGBoost uses γ (gamma) as a **pruning parameter**:

- Grow the tree to `max_depth` first
- Then prune **bottom-up**: if $\text{Gain} - \gamma \leq 0$, remove the split
- Higher $\gamma \Rightarrow$ more pruning $\Rightarrow$ simpler trees



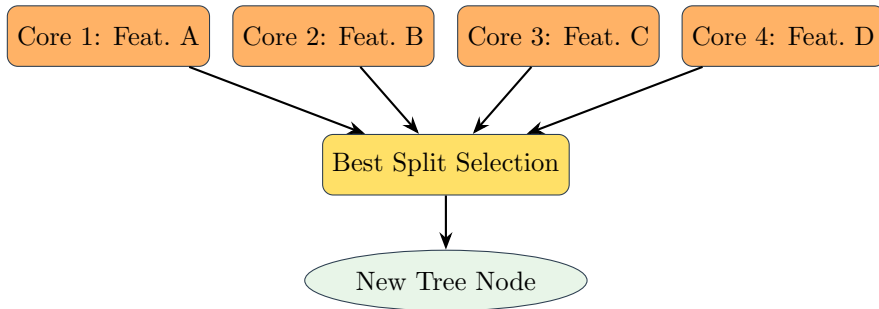
Effect of Regularization Parameter λ



As λ increases: lower similarity scores, smaller leaf outputs, **less sensitive to individual observations** $\Rightarrow$ less overfitting.

Parallel Processing in XGBoost

Key Insight: XGBoost parallelizes the **split finding**, not the tree building.



Each core evaluates splits for one feature in parallel

LightGBM

What is LightGBM?

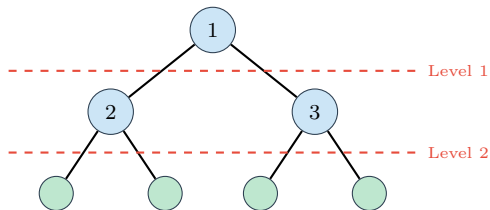
LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework by **Microsoft Research**, designed for speed and memory efficiency.

Key Innovations

- **Leaf-wise Tree Growth:** Grows leaf with max loss reduction (vs. level-wise)
- **GOSS:** Gradient-based One-Side Sampling
- **EFB:** Exclusive Feature Bundling for sparse features
- **Histogram-based Splits:** Buckets continuous features
- **Native Categorical Support:** No manual encoding needed

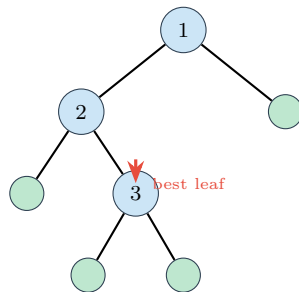
Leaf-wise vs Level-wise Tree Growth

Level-wise (XGBoost)



Splits **all** nodes at each level

Leaf-wise (LightGBM)



Splits leaf with **max** loss reduction

Leaf-wise can achieve lower loss per split but may overfit on small datasets.

Gradient-based One-Side Sampling (GOSS)

Idea: Instances with **large gradients** contribute more to information gain. Keep all of them; randomly sample among small-gradient instances.

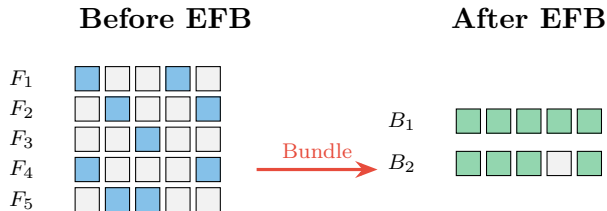
- 1 Sort instances by $|g_i|$ (absolute gradient)
- 2 Keep top $a \times 100\%$ instances (large gradients) $\rightarrow$ set A
- 3 Randomly sample $b \times 100\%$ from the rest $\rightarrow$ set B
- 4 Amplify sampled instances by factor $\frac{1-a}{b}$ when computing gain

$$\text{Gain}_{\text{GOSS}} \propto \left[\sum_{i \in A_l} g_i + \frac{1-a}{b} \sum_{i \in B_l} g_i \right]^2 - \text{similar for right}$$

Exclusive Feature Bundling (EFB)

Problem: High-dimensional sparse data has many features that are rarely non-zero simultaneously.

Solution: Bundle mutually exclusive features together.



Reduces feature dimension $\Rightarrow$ faster training

Features that are rarely non-zero simultaneously get merged into a single bundle using a **graph coloring** approach.

CatBoost

What is CatBoost?

CatBoost (Categorical Boosting) is a gradient boosting library by **Yandex**, designed to handle categorical features automatically with minimal tuning.

Key Innovations

- **Automatic Categorical Handling:** No manual encoding required
- **Ordered Boosting:** Prevents prediction shift / target leakage
- **Oblivious (Symmetric) Trees:** Same split at each level
- **Robust to Overfitting:** Built-in overfitting detection
- **Fast Inference:** No preprocessing pipeline at prediction time

Target Statistics for Categorical Features

Problem: How to encode categorical feature value c without target leakage?

CatBoost uses **Ordered Target Statistics**:

$$\hat{x}_{k,i} = \frac{\sum_{j=1}^{p-1} \mathbf{1}[x_{k,\sigma_j} = c] \cdot y_{\sigma_j} + \alpha}{\sum_{j=1}^{p-1} \mathbf{1}[x_{k,\sigma_j} = c] + \alpha/\text{prior}}$$

where:

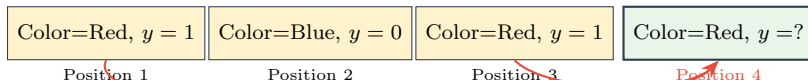
- σ = random permutation of training data
- p = position of current example in permutation
- α = smoothing parameter, prior = average target

Key Principle

Only examples **before** the current one in the permutation are used $\Rightarrow$ **no target leakage!**

Ordered Target Statistics – Visual Example

Permuted Training Data



Target statistic for “Red” at position 4:

$$\hat{x} = \frac{1 + 1 + \alpha}{2 + \alpha/\text{prior}} = \frac{2 + \alpha}{2 + \alpha/\text{prior}}$$

Only uses examples that appear **before** the current example in permutation — no target leakage!

Ordered Boosting

Standard boosting suffers from **prediction shift**: residuals are computed using a model that has already seen the training example.

CatBoost's solution — Ordered Boosting:

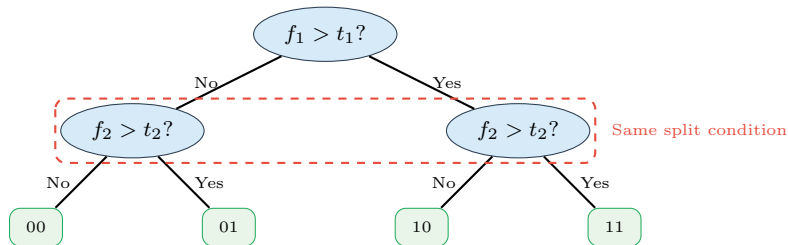
- For each sample x_i , train a model M_i on $\{x_j : j < i \text{ in permutation } \sigma\}$
- Compute residual: $r_i = y_i - M_i(x_i)$
- The model used for x_i has **never seen** x_i during training

Result

This produces **unbiased residuals**, leading to better generalization and less overfitting — especially on small datasets.

Oblivious (Symmetric) Decision Trees

CatBoost uses **oblivious trees**: same splitting criterion at each depth level.



Path to leaf: $(f_1(x) > t_1, f_2(x) > t_2, \dots, f_d(x) > t_d)$

Advantages: Less overfitting, faster scoring (lookup table), GPU-friendly.

Comprehensive Comparison

Algorithm Comparison (Part 1)

Aspect	GBM	XGBoost	LightGBM	CatBoost
Developer	J. Friedman	T. Chen	Microsoft	Yandex
Core Innovation	Sequential boosting + gradient descent	Regularized boosting + system optimizations	Leaf-wise growth + sampling	Categorical handling + ordered boosting
Tree Growth	Level-wise	Level-wise + pruning	Leaf-wise	Level-wise, oblivious
Optimization	1st-order gradients	2nd-order (Newton)	1st-order + sampling	1st-order + ordered boosting
Reg.	Learning rate only	L1, L2, γ , λ , α	L1, L2, dropout	Ordered boosting

Algorithm Comparison (Part 2)

Aspect	GBM	XGBoost	LightGBM	CatBoost
Categorical	Manual encoding	Manual encoding	Some native support	Full automatic
Missing Values	Manual handling	Auto (optimal dir.)	Automatic	Automatic
Training Speed	Baseline	Fast (parallel)	Fastest	Slower
GPU Support	No	Yes (CUDA)	Yes (OpenCL / CUDA)	Yes (CUDA)
Overfitting Risk	Moderate	Lower	Higher (leaf-wise)	Lowest
Best For	Learning	Competitions	Large datasets	Categoricals

Selection Guidelines

Gradient Boosting (GBM)

Learning, understanding fundamentals, simple datasets

XGBoost

Competitions, well-preprocessed data, maximum accuracy

LightGBM

Large datasets, speed is critical, memory constraints

CatBoost

Many categoricals, robust out-of-the-box, minimal preprocessing

Key Takeaways

Summary

Regression

- Start with **average**
- Build trees on residuals
- Scale by learning rate α

Hyperparameters

- **Learning Rate:** 0.1 typical
- **Tree Size:** 8–32 leaves
- **Stopping:** Max trees or convergence

Classification

- Start with **log-odds**
- Build trees on probability residuals
- Use special output formula

Core Insight

Taking lots of **small steps** in the right direction reduces overfitting and variance.

Thank You!

Questions?